Many modern computers may have 4 independent hardware processors, but they will have twice as many “virtual” processors via something called *hyperthreading*. Hyperthreading is a performance trick some processors use that can allow two processes to run essentially simultaneously on one physical processor. It is important not to create more processes than there are CPUs on your machine, as the additional processes will essentially function as threads, and the CPU will have to rapidly switch between processes.

In listing 5.1 we set up a processor pool of 8 processes with `mp.Pool(8)`, and then we used `pool.map` to distribute the square function across the 8 pieces of data. You can see we get a list of 8 arrays with all their elements squared, just as we wanted. Processes will return as soon as they’re complete, so the order of the elements in the returned list may not always be in the order they were mapped.

We’re going to need a bit more control over our processes than a processor pool allows, so we will create and start a bunch of processes manually.

Listing 5.2. Manually starting individual processes

```
def square(i, x, queue):
    print("In process {}".format(i,))
queue.put(np.square(x))
processes = []                          1
queue = mp.Queue()                      2
x = np.arange(64)                       3
for i in range(8):                      4
    start_index = 8*i
    proc = mp.Process(target=square,args=
(i,x[start_index:start_index+8],
                      queue))
    proc.start()
    processes.append(proc)

for proc in processes:                  5
    proc.join()

for proc in processes:                  6
proc.terminate()
results = []
while not queue.empty():                7
    results.append(queue.get())
>>> results
```